Holdout

Offers only where they change the outcome.

Bot @navpreet_offers_bot
Dashboard offer-engine-nu.vercel.app
Repo github.com/Navkaur-codes/Offer-Engine

Most discount engines send 20% off to everyone and count every sale that follows as a win. But many of those customers would have bought anyway, so the discount just gave away margin, and some customers buy less when pestered. **Holdout decides, for each customer and moment, whether an offer will actually change what they do, and only then which offer and when.** It delivers the offer through a Telegram bot and gives marketers a dashboard to set limits and prove the result. It is named after the group that deliberately gets nothing, which is how we measure what offers really caused.

+₹91,760	₹14,916	100%	~2 s
incremental revenue vs the holdout, 12 months	margin saved by not discounting sure-things	of offers re-checked by the same rule validator before sending	per AI decision; ~25 ms when the rules decide

How one decision is made

Event	Pre-gate	Policy	AI agent	Validator	Sent + logged

- **Rules set the limits; AI personalises inside them.** Marketers write limits as sentences. A Groq agent (gpt-oss-120b, read-only tools, strict JSON, 20 s) picks and words the offer. **The pre-gate, the agent's own self-check and the final validator all call the same** `evaluate()`, so they can never disagree. A property test pushes thousands of random decisions through it, and nothing outside the limits passes.
- **Never silent.** Timeout, rate limit, invalid JSON, two failed validations or an exhausted AI budget hand over to a deterministic fallback that picks the offer with the best expected incremental profit.
- **Never unexplained.** Every decision, including blocked, held-out and "no offer", is logged with why this customer, why this offer, the alternatives rejected, the rules applied and who decided. Explanations are built from numbers and rule results, **never from the AI's own words**, so an invented reason can't reach anyone.
- **Layered rules**, Global → Festival → Persona → Individual, where each layer can only tighten: margin guard, frequency and budget caps, quiet hours 21:00–09:00 IST, no discount within 48 h of a purchase, and 10 persona policies assigned live from scores (VIP: no % discounts; deal hunter: ≤10% over ₹999; high returner: points only; **sleeping dog: never contacted**).

Incrementality first: "don't pay people who'd buy anyway"

- Every synthetic customer hides a response type (persuadable, sure-thing, lost cause, sleeping dog) that the engine must infer from behaviour. A **T-learner uplift model**, trained on past campaigns with a 10% control group, predicts how much each offer type changes each customer's chance to buy.
- **Expected incremental profit** = uplift × typical basket × category margin − offer cost. Only offers that pay for themselves are allowed, and **zero-cost perks come first** (cart nudge, early access, 2× points).
- A **10% holdout** gets decisions but no messages. The Experiments page compares each campaign with its own control: flat ₹ off returned 1.9×, while **blanket free shipping lost ₹24,320**, because it mostly reached people who'd have bought anyway. That is the case for targeting, in our own data.

Culturally safe, festival-aware

26 festivals and sales (Sep 2025 – Jan 2027) with phases and hero categories; the judging week sits in the Karwa Chauth → Diwali run-up. **Religion is never inferred from a name:** greetings go only to customers who opted in, the text comes from the festivals table (never the AI), and restricted promotions are blocked by the validator. Consent is immediate (`/pause`, `/stop`), and no proactive message is sent at night.

Try it in 5 minutes

Where	Do this	You'll see
Telegram	Open @navpreet_offers_bot, send /start, answer 3 taps	A profile built for you and a welcome perk as a picture card
Telegram	/simulate cart, then tap **?** Why this?	A real offer through the full pipeline, then why you, why this offer and the rules checked
Telegram	/simulate purchase, then /simulate cart	No offer: the 48-hour rule blocks it, and the bot says so
Dashboard	Overview, then Decisions	Your decisions arrive live, tagged AI agent or Rules fallback, each with its Pipeline Lens
Dashboard	Playground → Cart abandoner → Run; toggle "force LLM failure"	Every step explained; the fallback still decides safely without the AI
Dashboard	Rules → change "Deal hunters get at most 10%" to 8%	Customers and budget affected before you save; versions with one-click revert

Holdout

Offers only where they change the outcome.

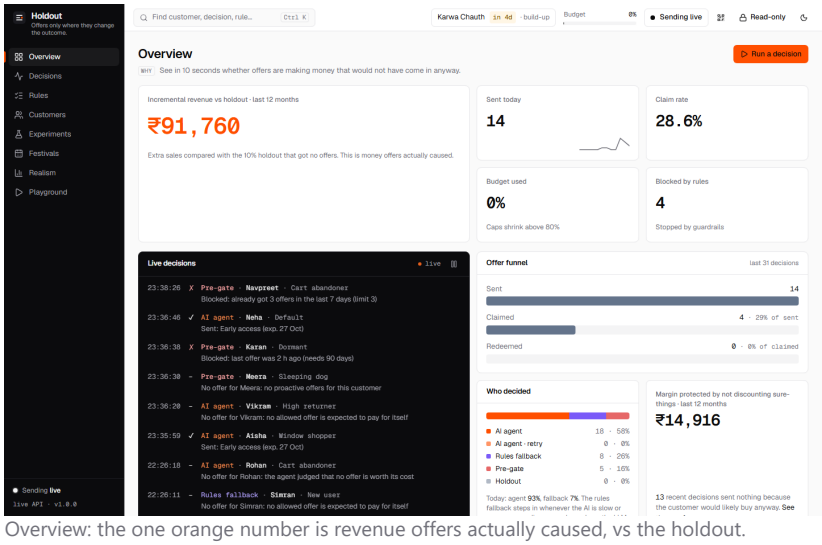
Bot @navpreet_offers_bot
Dashboard offer-engine-nu.vercel.app
Repo github.com/Navkaur-codes/Offer-Engine

What's in it

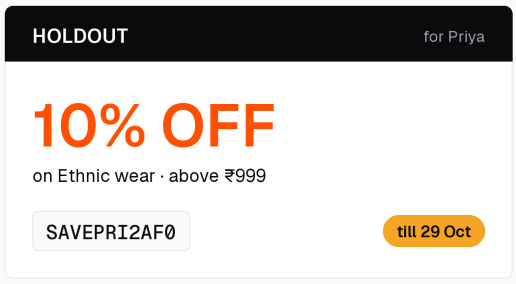
Part	What it does
Telegram bot	`/start`: 3 taps build a persona with 6 months of synthetic history and a welcome perk. `/simulate cart · browse · purchase · inactive · festival · payday` fire real events through the full pipeline. Offers arrive as **picture cards** with ✅ Claim · ⏰ Remind me · 🙈 Not for me · **?** Why this? Plus `/why /history /profile /offers` and admin `/summary /killswitch /campaign /impersonate`.
Customer Lab	A Telegram group where 10 synthetic personas, voiced by "Lab Shopper" bots, shop and react in character. Judges watch different personas get different offers under different rules, and some get nothing on purpose. `/lab date` time-travels to any festival phase.
Dashboard	Overview (one orange hero KPI, live decision stream) · Decisions with the **Pipeline Lens** (Pre-gate → Policy → Agent → Validator → Sent, each ✓/✗) and agent trace · **Rules as sentences** with impact preview, versions and revert · Customers · Experiments · Festivals · Realism · **Playground** with a "force LLM failure" toggle · campaign dry-run · kill switch and AI budget always visible · ⌘K.
Scheduler	Abandoned-cart nudges, a 09:05 flush of offers held during quiet hours, points-expiry reminders and a 21:00 digest whose numbers are computed in code (the AI may only reword them).
Synthetic data	12 months simulated day by day: funnels, lognormal baskets, seasonality, payday and festival lifts, churn, returns and messy fields. Benchmarks in range: top 20% ≈ 75% of revenue, cart abandonment ≈ 69%.

Judging criteria → evidence

Criterion	Evidence
Core functionality	Live for anyone: bot → harness → evaluate() → Telegram card → log → dashboard; AI with a guaranteed rules fallback; every decision explained.
Data realism	Day-by-day behaviour simulation; Realism page compares each statistic with a cited benchmark band (✓/✗); data dictionary in the repo.
Usability for marketers	Rules are sentences with editable numbers, an impact preview before saving and one-click revert; "why this page exists" on every page; kill switch and dry-runs one click away.
Design	"Signal" system: cool monochrome, one orange accent for what matters, Geist type, Pipeline Lens, terminal-style stream; no gradients or pie charts; status never by colour alone; Lighthouse accessibility 100.
Advanced functionality	Tool-using LLM agent in a harness; T-learner uplift + holdout; token budget guard (120B → 20B → rules); top-5 agent batching; multi-bot Customer Lab; time-travel.
Business relevance	Incrementality-first targeting that saves margin; zero-cost perks first; budget pacing; festival calendar; consent and quiet hours.



Overview: the one orange number is revenue offers actually caused, vs the holdout.



Offer card, drawn by the server in the same design system and sent as a photo.

Built with FastAPI · SQLModel · APScheduler · python-telegram-bot · Groq · scikit-learn · Pillow · React · TypeScript · Tailwind · Recharts. Hosted free on Render, Neon Postgres and Vercel. **362 backend tests** (property test on the validator, AI failure cases, every fallback message validated).

How we built it: five Claude Code sessions in parallel worktrees, coordinated by CLAUDE.md (non-negotiables), shared contracts with stubs, and a "detective" trap list; an integrator merged them at 5 gates, each with tests, a smoke test and a live check.